Query Processing in DBMS

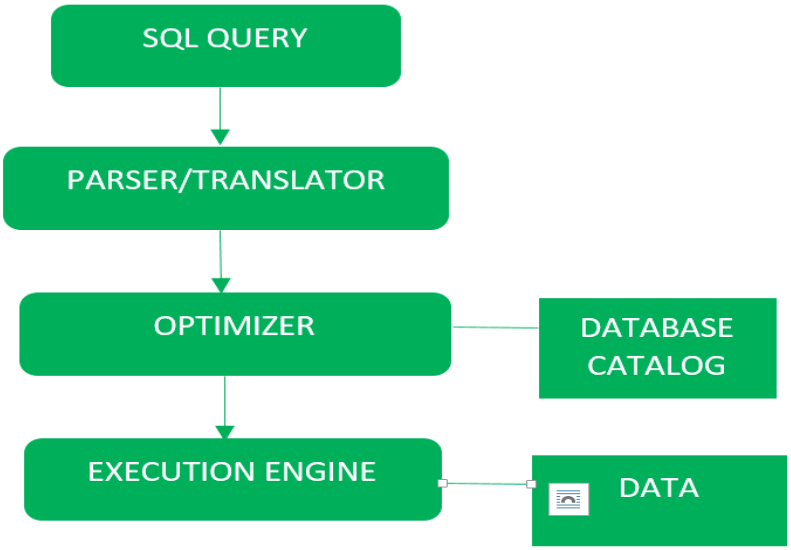
Query Processing is the activity performed in extracting data from the database. In query processing, it takes various steps for fetching the data from the database. The steps involved are:

1. Parsing and translation
2. Optimization
3. Evaluation
4. The query processing works in the following way:

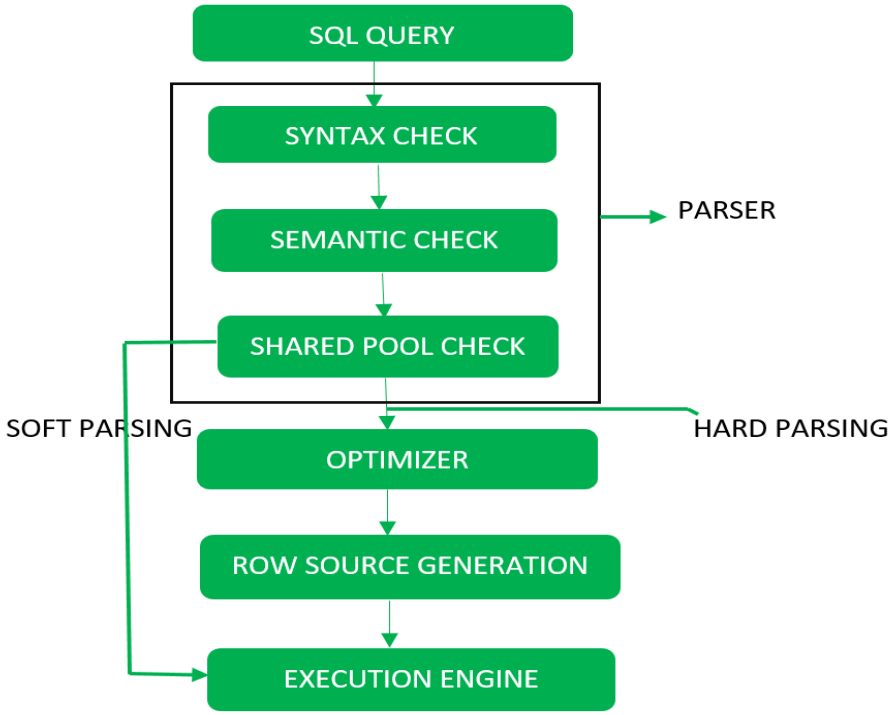
Parsing and Translation

As query processing includes certain activities for data retrieval. Initially, the given user queries get translated in high-level database languages such as SQL. It gets translated into expressions that can be further used at the physical level of the file system. After this, the actual evaluation of the queries and a variety of query -optimizing transformations and takes place. Thus, before processing a query, a computer system needs to translate the query into a human-readable and understandable language. Consequently, SQL or Structured Query Language is the best suitable choice for humans. But it is not perfectly suitable for the internal representation of the query to the system. Relational algebra is well suited for the internal representation of a query. The translation process in query processing is similar to the parser of a query. When a user executes any query, for generating the internal form of the query, the parser in the system checks the syntax of the query, verifies the name of the relation in the database, the tuple, and finally the required attribute value. The parser creates a tree of the query, known as 'parse-tree.' Further, translate it into the form of relational algebra. With this, it evenly replaces all the use of the views when used in the query.

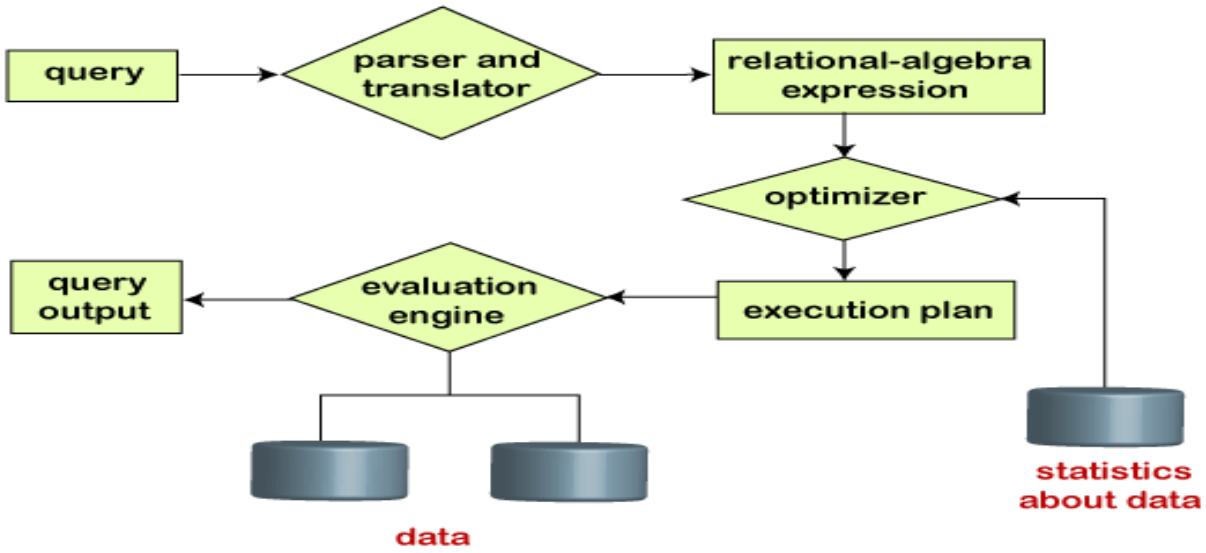
The Block Diagram of Query Processing is as:



A detailed Diagram is drawn as:



Thus, we can understand the working of a query processing in the below-described diagram:



Steps in query processing

Suppose a user executes a query. As we have learned that there are various methods of extracting the data from the database. In SQL, a user wants to fetch the records of the employees whose salary is greater than or equal to 10000. For doing this, the following query is undertaken:

select emp_name from Employee where salary>10000;

Thus, to make the system understand the user query, it needs to be translated in the form of relational algebra. We can bring this query in the relational algebra form as:

- $\sigma_{salary>10000}$ (π_{salary} (Employee))
- π_{salary} ($\sigma_{salary>10000}$ (Employee))

After translating the given query, we can execute each relational algebra operation by using different algorithms. So, in this way, a query processing begins its working.

Evaluation

For this, with addition to the relational algebra translation, it is required to annotate the translated relational algebra expression with the instructions used for specifying and evaluating each operation. Thus, after translating the user query, the system executes a query evaluation plan.

Query Evaluation Plan

- In order to fully evaluate a query, the system needs to construct a query evaluation plan.
- The annotations in the evaluation plan may refer to the algorithms to be used for the particular index or the specific operations.

- Such relational algebra with annotations is referred to as **Evaluation Primitives**. The evaluation primitives carry the instructions needed for the evaluation of the operation.
- Thus, a query evaluation plan defines a sequence of primitive operations used for evaluating a query. The query evaluation plan is also referred to as **the query execution plan**.
- A **query execution engine** is responsible for generating the output of the given query. It takes the query execution plan, executes it, and finally makes the output for the user query.

Optimization

- The cost of the query evaluation can vary for different types of queries. Although the system is responsible for constructing the evaluation plan, the user does need not to write their query efficiently.
- Usually, a database system generates an efficient query evaluation plan, which minimizes its cost. This type of task performed by the database system and is known as Query Optimization.
- For optimizing a query, the query optimizer should have an estimated cost analysis of each operation. It is because the overall operation cost depends on the memory allocations to several operations, execution costs, and so on.

Finally, after selecting an evaluation plan, the system evaluates the query and produces the output of the query.

Evaluation of Expressions

In our previous sections, we understood various concepts in query processing. We learned about the query processing steps, selection operations, and also several types of algorithms used for performing the join operation with their cost estimations.

We are already aware of computing and representing the individual relational operations for the given user query or expression. Here, we will get to know how to compute and evaluate an expression with multiple operations.

For evaluating an expression that carries multiple operations in it, we can perform the computation of each operation one by one. However, in the query processing system, we use two methods for evaluating an expression carrying multiple operations. These methods are:

1. Materialization
2. Pipelining

Let's take a brief discussion of these methods.

Materialization

In this method, the given expression evaluates one relational operation at a time. Also, each operation is evaluated in an appropriate sequence or order. After evaluating all the operations, the outputs are materialized in a temporary relation for their subsequent uses. It leads the materialization method to a disadvantage. The disadvantage is that it needs to construct those temporary relations for materializing the results of the evaluated operations, respectively. These temporary relations are written on the disks unless they are small in size.

Pipelining

Pipelining is an alternate method or approach to the materialization method. In pipelining, it enables us to evaluate each relational operation of the expression simultaneously in a pipeline. In this approach, after evaluating one operation, its output is passed on to the next operation, and the chain continues till all the relational operations are evaluated thoroughly. Thus, there is no requirement of storing a temporary relation in pipelining. Such an advantage of pipelining makes it a better approach as compared to the approach used in the materialization method. Even the costs of both approaches can have subsequent differences in-between. But, both approaches perform the best role in different cases. Thus, both ways are feasible at their place.

We have described and discussed the materialization as well as pipelining method deeply in our next sections one by one.

Measures of Query Cost in DBMS

Query Cost is a cost in which the enhancer considers what amount of time your query will require (comparative with absolute clump time). Then the analyser attempts to pick the most ideal query plan by taking a glance at your inquiry and insights of your information, attempting a few execution designs, and choosing the most inexpensive of them.

The measures of query cost in DBMS can be done by creating a framework that can make numerous designs for an inquiry. It tends to be finished by the means of contrasting every conceivable arrangement as far as their assessed cost. For working out the net assessed cost of any arrangement, the expense of every activity inside an arrangement ought to be set in a deterministic and consolidated cost to get the net assessed cost of the query assessment plan.

Example: We utilize the number of square exchanges that is basically the block from the disk and the quantity of the disk seeks to appraise the expense of a query assessment plan. Assuming that the disk subsystem takes a normal of tT seconds to move a square of information and has a normal block access time (disk lookup

time in addition to rotational idleness) of tS seconds, then, at that point, an activity that moves **b** obstructs and performs **S** looks for would take b ∗ tT + S ∗ tS seconds. The upsides of tT and tS should be aligned for the disk framework utilization, however, normal qualities for top-end disk today would be tS = 4 milliseconds and tT = 0.1 milliseconds, expecting a 4-kilobyte block size and an exchange pace of 40 megabytes each second.

```
tT – time to transfer one block

tS – time for one to seek

Cost for b block transfers plus S seeks

b * tT + S * tS
```

The expense assessment of a query assessment plan is determined by keeping in mind the different assets that follow as:

1. The number of disk accesses.
2. Time of Execution taken by the CPU to execute a query.
3. The involved Communication costs in either distributed or parallel database systems.

To gauge the expense of a query assessment plan, utilizing the number of blocks moved from the disk, and the quantity of disk seeks looks for. For the most part, for assessing the expense, we must consider the most pessimistic scenario that could occur which is the worst-case scenario. The clients accept that at first, the information is perused from the disk as it were. However, there should be an opportunity that the data is now present in the principle memory also called the main memory. Notwithstanding, the clients generally disregard this impact, and because of this, the genuine expense of execution comes out lesser than the assessed esteem.

The reaction time, i.e., the time that is expected so as to execute the arrangement could be utilized for assessing the expense of the query assessment plan. Yet, because of the accompanying reasons, it becomes hard to work out the reaction time without really executing the query assessment plan. As soon as the query starts the process of execution, the time of reaction becomes reliant upon the substance put away in support. In any case, this data is so hard to recover that when the query is in its upgraded mode, or it isn't accessible.

Whenever there is a framework with the various presence of disks available, the time of reaction relies upon a cross-examination that is ready "what is the way that accesses gets to be circulated among the disk available?". It is hard to gauge without having itemized information on the information format present over the disk. Thus, rather than limiting the time of reaction for any query assessment plan, the analysers observe that it is better to lessen the wholesome asset utilization of the query plan. Along with these lines to appraise the expense of a query assessment plan, it is great to limit the assets utilized for getting to the disk or utilization of the additional assets.

Query Execution Plan in SQL

An **execution plan in SQL** is a detailed plan that outlines the steps that the database management system (DBMS) will take to execute a query. It is a crucial component of query optimization, as it helps the DBMS determine the most efficient way to retrieve data from the database.

Here, we will learn all about **query execution plans in SQL**, understand the **types of query execution plans**, **how to create query plans**, and at last **how to save query plans** in SQL Server.

What is an Execution Plan?

An **SQL query execution plan** (also known as a **query plan**) is a sequence of steps used by a relational database management system (RDBMS) to access data efficiently when executing a query. The query plan is also referred to as the **SQL Server execution plan**.

The **query optimizer** generates the SQL Server execution plan or query plan. Its goal is to generate an optimal economic query plan. Multiple execution plans are generated by the query processing engine after query execution, and from those generated execution plans, a plan with the best performance is selected.

The execution plans are stored in the **plan cache** memory location, from where they can be reused. Execution plans are available in 3 forms in the SQL server: **XML plans**, **Graphical plans**, and **Text plans**.

Types of Execution Plan in SQL

There are **two types** of Execution Plans in SQL:

- **Estimated Execution Plan**
- **Actual Execution Plan**

1. Actual Execution Plan

An **actual execution plan is the SQL Server** query plan that is generated after a query has been executed. It contains runtime information, such as actual resource usage metrics and any runtime warnings that occurred during execution.

An actual execution plan displays the actual query execution plan that the SQL Server Database Engine used to execute the queries. You can find the information about actual number of rows processed, resource utilization, and other relevant statistics

2. Estimated Execution Plan

An estimated execution plan is a prediction made by the SQL Server query optimizer regarding the steps it expects to take when executing a query. The estimated execution plan is generated **before** the query is executed.

Estimated execution plans are created during query compilation, before the query is actually executed. These plans are based on statistical information about the database schema, indexes, and data distribution.

An estimated execution plan provides information about the expected sequence of operations. It includes details about the logical and physical operators involved (e.g., scans, joins, sorts).

Generating and Saving Execution Plans in SQL Server Management Studio

Before and after the execution of the query, the execution plans in SQL Server. Actual and estimated execution plans can be achieved by the given steps:

Generation of Actual Execution Plans

The actual execution plan can be achieved in the following ways in SQL Server:

1. After completely writing the query, Press Ctrl+M, and the actual execution plan will be generated.
2. Go to the query window and right-click on it, then click on the context menu and select '*Display Actual Execution Plan*'.
3. Or the '*Display Actual Execution Plan*' icon can be directly selected from the toolbar.

Generation of Estimated Execution Plans

An estimated execution plan can be achieved using the following ways in SQL Server:

1. After completely writing the query, Press Ctrl+L, and the plan will be generated.
2. Go to the query window and right-click on it, then click on the context menu and select "*Display Estimated Execution Plan* ".
3. Or the "*Display Estimated Execution Plan*" icon can be directly selected from the toolbar.

How to save a Query Execution plan?

One has to save the query plan after interpreting the plan produced by the query. SQL Server Management Studio has an extension of ". sqlplan" for saving the plan in the system.

Steps to save an execution plan:

1. Go to the plan window and right-click.
2. Click on 'Save Execution Plan As'.
3. Click on the folder or location where you want to save the execution plan, then give the name to the plan and click on 'Save'.

Materialization v/s Pipelining

Materialization

- Materialized evaluation walks the parse or expression tree of the relational algebra operation, and performs the innermost or leaf-level operations first
- The intermediate result of each operation is materialized — an actual, but temporary, relation —and becomes input for subsequent operations.
- The cost of materialization is the sum of the individual operations plus the cost of writing the intermediate results to disk — a function of the blocking factor (number of records per block) of the temporaries.
- The problem with materialization is that — lots of temporary files, lots of I/O.

Pipelining

- With pipelined evaluation, operations form a queue, and results are passed from one operation to another as they are calculated, hence the technique's name.
- With pipelined evaluation, operations form a queue, and results are passed from one operation to another as
  they are calculated, hence the technique's name.
- General approach: restructure the individual operation algorithms so that they take streams of tuples as both input and output.
- Limitation: General approach: restructure the individual operation algorithms so that they take streams of tuples as both
  input and output.
- So, for instance, algorithms that require sorting can only use pipelining if the input is already sorted beforehand, since sorting by nature cannot be performed until all tuples to be sorted are known.

Materialization in Query Processing

In the previous, we took a brief introduction about materialization and how to evaluate multiple operations of an expression.

Materialization is an easy approach for evaluating multiple operations of the given query and storing the results in the temporary relations. The result can be the output of any join condition, selection condition, and many more. Thus, materialization is the process of creating and setting a view of the results of the evaluated operations for the user query. It is similar to the cache memory where the searched data get settled temporarily. We can easily understand the working of materialization through the pictorial representation of the expression. An operator tree is used for representing an expression.

The materialization uses the following approach for evaluating operations of the given expression:

- In the operator tree, we begin from the lowest-level operations (at the bottom of the tree) in the expression. The inputs to the lowest level operations are stored in the form of relations in the database. **For example,** suppose we want to fetch the name of the student as 'John' from the 'Student' relation.
  The relation expression will be:
  $\sigma_{name="John"}$ **(Student)**
  In this example, there is only one operation of selecting the name from the given relation. Also, this operation is the lowest-level operation. So, we will begin by evaluating this selection operation.
- Now, we will use an appropriate algorithm which is suitable for evaluating the operation. Like in our example, we will use an appropriate selection algorithm for retrieving the name from the Student relation.
- Then, store the result of the operation in the temporary relations.
- We use these temporary relations for evaluating the next-level operation in the operator tree. The result works as an input for every next level up in the tree.

- Repeat these steps until all operators at the root of tree will be evaluated, and the final result of the expression will be generated.

We also call the described evaluation as Materialized evaluation because the result of one operation is materialized and used in the evaluation of next operation and so on.

Cost Estimation of Materialized Evaluation

The process of estimating the cost of the materialized evaluation is different from the process of estimating the cost of an algorithm. It is because in analyzing the cost of an algorithm, we do not include the cost of writing the results on to the disks. But in the evaluation of an expression, we not only compute the cost of all operations but also include the cost of writing the result of currently evaluated operation to disk.

To estimate the cost of the materialized evaluation, we consider that results are stored in the buffer, and when the buffer fills completely, the results are stored to the disk.

Let, a total of b_r number of blocks are written. Thus, we can estimate b_r as:

$b_r = n_r/f_r$.

Here, n_r is the estimated number of tuples in the result relation r and fr is the number of records of relation r that fits in a block. Thus, fr is a blocking factor of the resultant relation r.

With this, we also need to calculate the transfer time by estimating the number of required disks. It is so because the disk head may have moved in-between the successive writes of the block. Thus, we can estimate:

Number of seeks = $\lceil b_r / b_b \rceil$

Here, b_b defines the size of the output buffer, i.e., measured in blocks.

We can optimize the cost estimation of the materialization process by using the concept of double buffering. Double buffering is the method of using two buffers, where one buffer executes the algorithm continuously, and the other is being written out. It makes the algorithm to execute more fastly by performing CPU activities parallel with I/O activities. We can also reduce the number of seeks by allocating the extra blocks to the output buffer and altogether writing out multiple blocks.

Pipelining in Query Processing

In the earlier section, we learned about materialization in which we evaluate multiple operations in the given expression via temporary relations. But, it leads to a drawback of producing a high number of temporary files. It makes the query-evaluation less

efficient. However, the evaluation of the query should be highly efficient in producing an effective output.

Here, we will discuss another method of evaluating the multiple operations of an expression that works more efficiently than materialization. Such a more efficient way is known as **Pipelining**. Pipelining helps in improving the efficiency of the query-evaluation by decreasing the production of a number of temporary files. Actually, we reduce the construction of the temporary files by merging the multiple operations into a pipeline. The result of one currently executed operation passes to the next operation for its execution, and the chain continues till all operations are completed, and we get the final output of the expression. Such type of evaluation process is known as **Pipelined Evaluation**.

Advantages of Pipeline

There are following advantages of creating a pipelining of operations:

- It reduces the cost of query evaluation by eliminating the cost of reading and writing the temporary relations, unlike the materialization process.
- If we combine the root operator of a query evaluation plan in a pipeline with its inputs, the process of generating query results becomes quick. As a result, it is beneficial for the users as they can view the results of their asked queries as soon as the outputs get generated. Else, the users need to wait for high-time to get and view any query results.

Pipelining vs. Materialization

Although both methods are used for evaluating multiple operations of expression, there are few differences between them. The difference points are described in the below table:

Pipelining	Materialization
It is a modern approach to evaluate multiple operations.	It is a traditional approach to evaluate multiple operations.
It does not use any temporary relations for storing the results of the evaluated operations.	It uses temporary relations for storing the results of the evaluated operations. So, it needs more temporary files and I/O.

It is a more efficient way of query evaluation as it quickly generates the results.	It is less efficient as it takes time to generate the query results.
It requires memory buffers at a high rate for generating outputs. Insufficient memory buffers will cause thrashing.	It does not have any higher requirements for memory buffers for query evaluation.
Poor performance if trashing occurs.	No trashing occurs in materialization. Thus, in such cases, materialization is having better performance.
It optimizes the cost of query evaluation. As it does not include the cost of reading and writing the temporary storages.	The overall cost includes the cost of operations plus the cost of reading and writing results on the temporary storage.